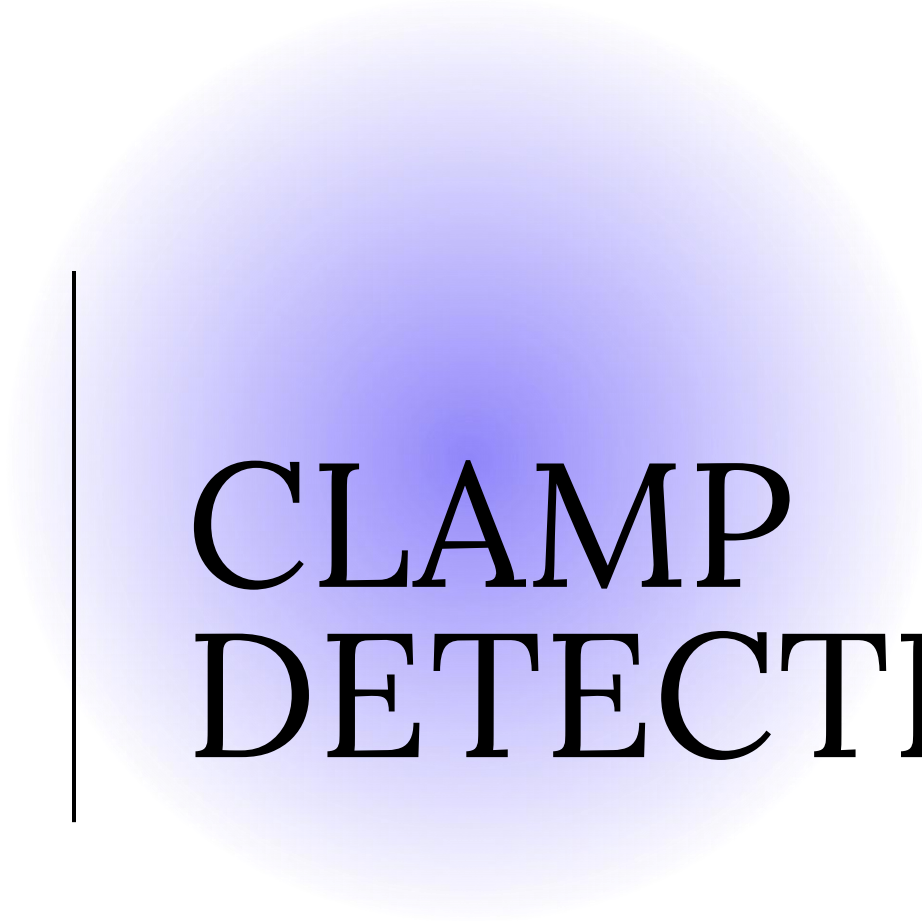
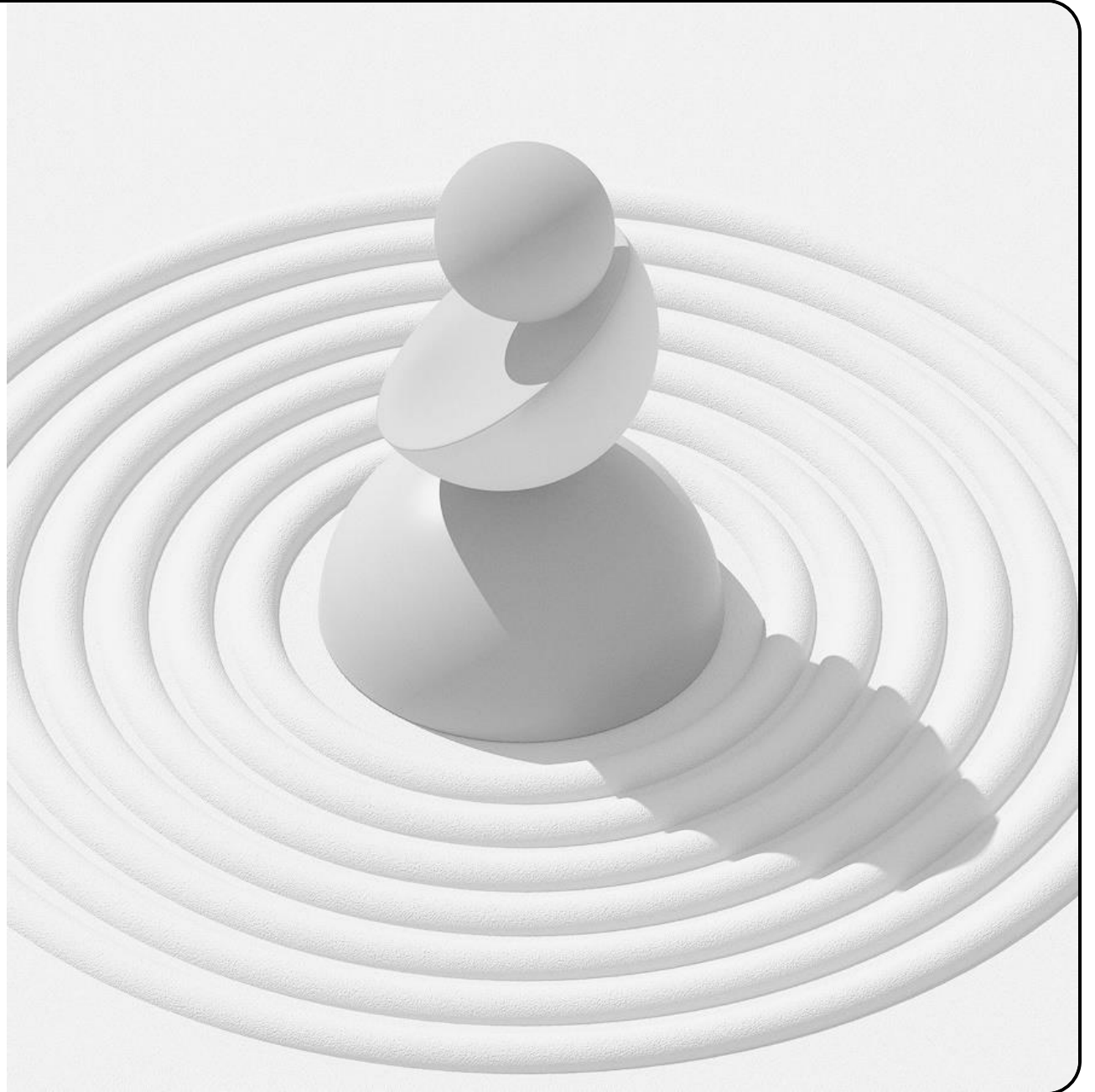




CLAMP DETECTION

AGENDA

- Dataset
- Model
- Evaluation
- Prediction



DATASET

- Used python script to extract frames out of the video
- Uploaded the frames to roboflow
- Image Annotation
- Created dataset from images

```
import cv2
import os

def extract_frames(video_path, output_folder):
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Could not open video.")
        return
    frame_count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        file_path = os.path.join(output_folder, f"frame_{frame_count}.jpg")
        cv2.imwrite(file_path, frame)

        frame_count += 1

    cap.release()
    print(f"Extraction complete. {frame_count} frames saved to '{output_folder}'.")

path = os.getcwd()
extract_frames(os.path.join(path, 'Task.mp4'), 'frames')
```

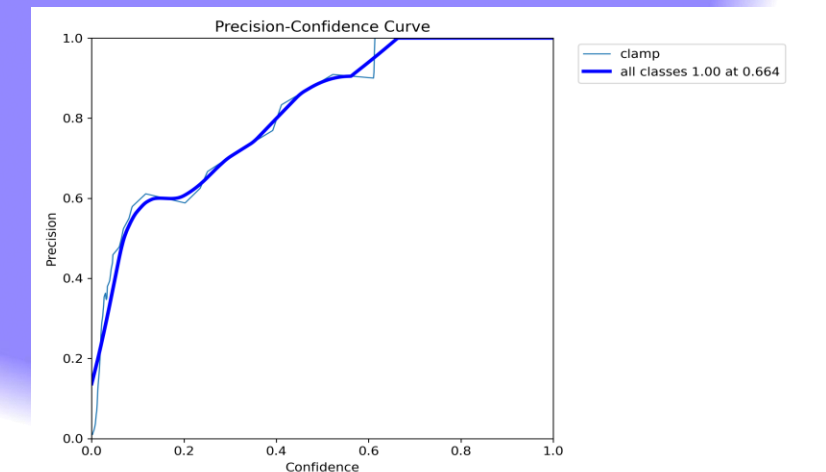
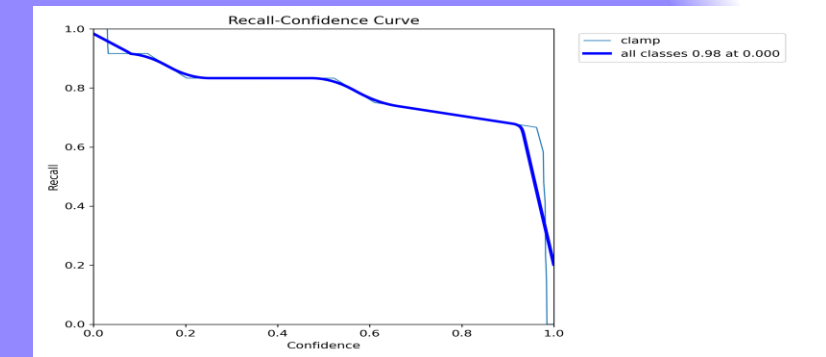
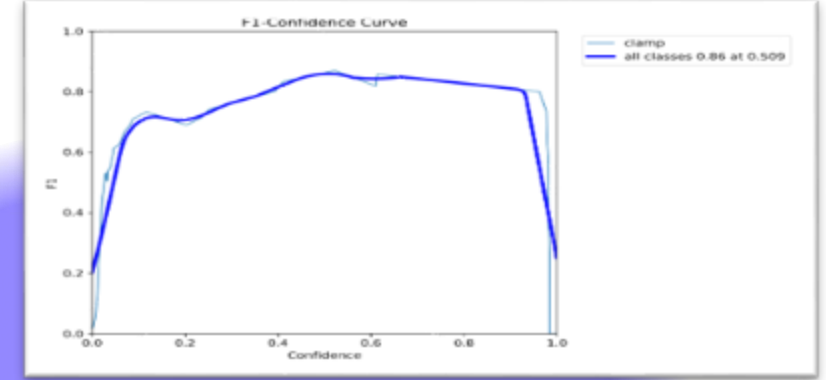
MODEL

- Upload the dataset from roboflow
- Upload the yolo model from ultralytics
- Fit the model to the dataset
- I tried different hyperparameters and settled for those

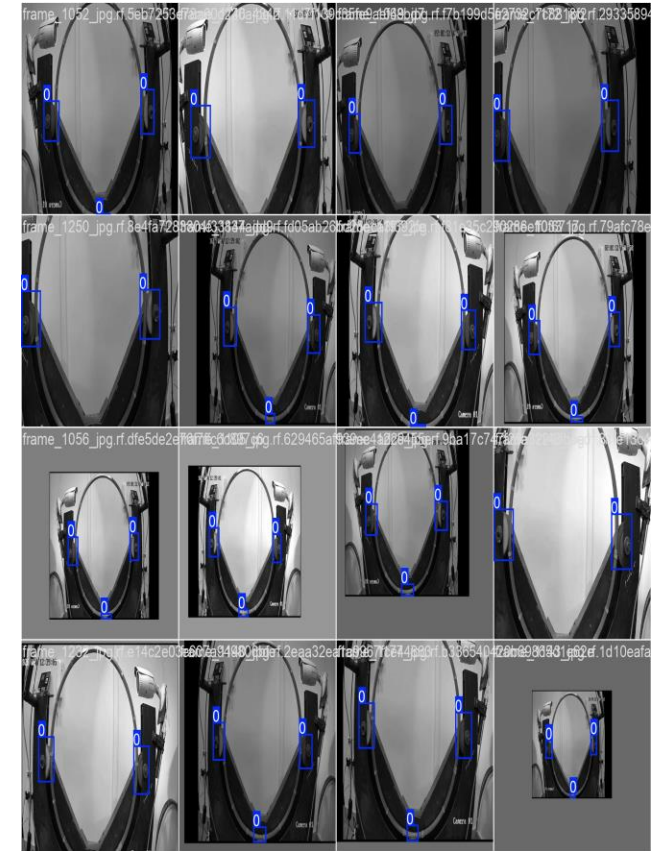
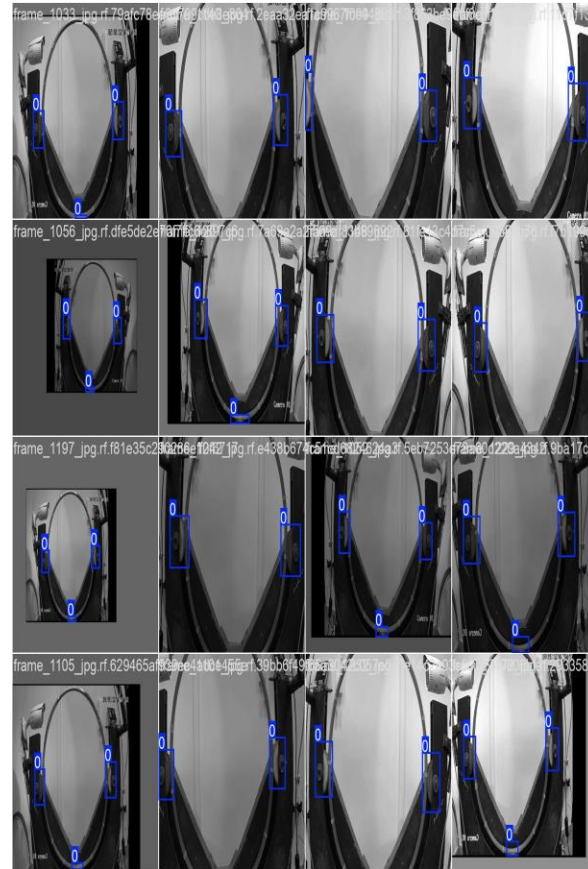
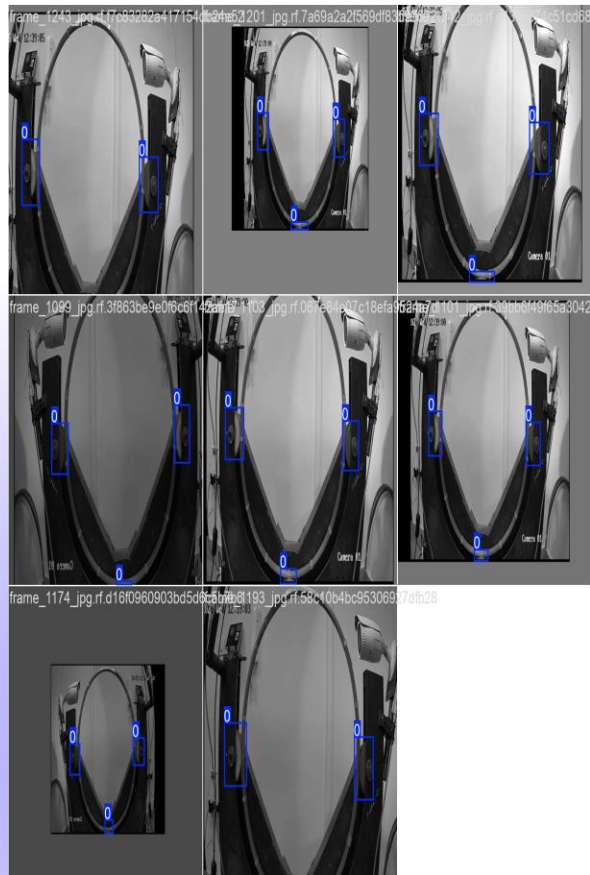
```
rf = Roboflow(api_key="bgZz4XsPKrewZX7Qhbab")
project = rf.workspace("ahmeds-workspace-slwao").project("clamp_detection-0fjwm")
version = project.version(2)
dataset = version.download("yolov11")
model = YOLO("yolo11m.pt")
results = model.train(data="clamp_detection-2/data.yaml", epochs=10, imgsz=640 )
```

EVALUATION

- The Model has high F1 score around 50% confidence
- For this model we need high recall because we want to make sure to detect all the clamps therefore we will set the confidence to be low when predicting so we can get higher recall



TRAINING INSIGHTS



PREDICTING

- We use this script to process the video and detect the clamps
- We go through the video frame by frame and we send each frame through the model
- We take the dimensions of the detected clamps from the results and we write new annotated output frame
- We set the conf to 10% to get higher recall and Overlap Threshold to zero

```
input_video = "/content/Task.mp4"
output_video = "/content/predict.mp4"

# Open input video
cap = cv2.VideoCapture(input_video)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_video, fourcc, fps, (width, height))
while True:
    ret, frame = cap.read()

    if not ret:
        break
    results = model(frame , conf =.1 , iou=0 )
    annotated_frame = results[0].plot()

    out.write(annotated_frame)
    # Press q to quit early
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
out.release()
cv2.destroyAllWindows()
```